龙芯 2K300/301 系列板卡使用手册-CANFD 通信

龙芯 2K300/301 开源库使用说明

CANFD 通信篇

一、CANFD 模块简介

CANFD（CAN with Flexible Data-rate）是传统 CAN 总线的高性能升级版本，专为解决传统 CAN 总线数据传输速率低、数据长度有限的痛点而设计，兼容传统 CAN 2.0 协议，同时大幅提升通信效率与数据容量，是龙芯 2K300/301 板卡实现工业控制、智能设备互联、车载通信等场景的核心通信模块。

龙芯 2K300/301 开源库提供了 LS_CANFD 封装类，底层已完成 LoongArch 架构深度适配，集成了 CANFD 接口初始化、数据发送、数据接收、回调配置等核心功能，开发者无需关注底层 socket 编程、波特率配置等细节，直接调用封装接口即可快速实现 CANFD 通信功能，大幅降低嵌入式通信开发门槛。

1.1 核心特性

- 支持可变速率通信，默认配置仲裁段 500kbps、数据段 2Mbps，可根据需求灵活调整
- 数据场长度扩展至 64 字节，相比传统 CAN（8 字节）大幅提升单次数据传输量
- 兼容传统 CAN 2.0 协议，可与传统 CAN 设备无缝通信，降低系统升级成本
- 提供 LS_CANFD 封装类，支持阻塞、异步信号、独立线程三种接收模式，适配不同应用场景
- 支持回调函数机制，可灵活处理接收数据，支持多任务协同，可与摄像头、ncnn 推理等模块联动
- 初始化自动完成接口配置，无需手动执行 ip link 等命令，简化开发流程

重要说明：LS_CANFD 类已完全适配龙芯 2K300/301 板卡的 CAN0、CAN1 两个接口，支持所有核心通信功能；如需使用自定义波特率或扩展帧通信，需参考相关配

置章节进行参数调整。

二、CANFD 库文件位置

龙芯 2K300/301 开源库中，CANFD 相关文件按固定目录结构存放，包含头文件、实现文件、示例文件，便于开发者调用、调试与二次开发。

2.1 头文件和实现文件

- 头文件：libraries/drv/inc/lq_canfd.hpp（包含 LS_CANFD 类定义、接口声明、枚举与结构体定义）
- 实现文件：libraries/drv/src/lq_canfd.cpp（包含 LS_CANFD 类接口的具体实现，底层适配 LoongArch 架构）

2.2 示例文件

示例文件位置：example/src/06_lq_canfd_demo.cpp

包含内容：涵盖独立线程模式、阻塞模式、数据帧结构体发送等常见使用场景的完整示例代码，可直接编译运行，开发者可根据实际需求修改使用。

三、LS_CANFD 类接口说明

LS_CANFD 类是龙芯开源库对 CANFD 底层 socket API 的封装，提供简洁统一的接口，支持 CANFD 接口初始化、数据发送、数据接收、回调配置等核心功能，调用便捷、稳定性高，适配龙芯 2K300/301 板卡的硬件特性。

3.1 宏定义与数据类型

【接口名称宏定义】

```c++
#define CAN0    "can0"  // 龙芯板卡 CAN3
#define CAN1    "can1"  // 龙芯板卡 CAN4
```

功能：定义龙芯 2K300/301 板卡的两个 CANFD 接口名称，开发者可直接使用宏定义调用接口，避免手动输入错误。

【接收模式枚举】

```c++
typedef enum {
    CANFD_MODE_BLOCKING = 0,  // 阻塞模式
    CANFD_MODE_ASYNC    = 1,  // 异步信号模式（会中断主线程 sleep）
    CANFD_MODE_THREAD   = 2   // 独立线程模式（推荐，不影响主线程）
} ls_canfd_rx_mode_t;
```

功能：定义 CANFD 数据接收的三种模式，开发者可根据应用场景选择合适的接收模式。

【数据帧结构体】

```c++
typedef struct {
    uint32_t can_id;                    // CAN ID（标准帧/扩展帧标识）
    uint8_t  len;                       // 数据长度（0~64 字节）
    uint8_t  data[CANFD_MAX_DATA_LEN];  // 数据缓冲区（最大 64 字节）
} ls_canfd_frame_t;
```

功能：存储 CANFD 数据帧的完整信息，包括 CAN ID、数据长度和数据内容，用于数据发送与接收。

【接收回调函数类型】

```c++
typedef std::function<void(const ls_canfd_frame_t &frame)>
ls_canfd_rx_callback_t;
```

功能：定义接收回调函数的类型，用于处理接收到的 CANFD 数据帧，适用于异步信号模式和独立线程模式。

3.2 构造与析构

【构造函数】

```c++
// 无参构造
ls_canfd();
```

```
// 有参构造（推荐使用）
ls_canfd(const std::string &ifname,
         ls_canfd_rx_mode_t rx_mode = CANFD_MODE_THREAD,
         ls_canfd_rx_callback_t _cb = nullptr);
```

参数说明：

- ifname：CANFD 接口名称，使用 CAN0 或 CAN1 宏定义
- rx_mode：接收模式，默认使用独立线程模式（CANFD_MODE_THREAD）
- _cb：接收回调函数，默认为空，可后续通过 set_rx_callback 函数设置

功能：创建 LS_CANFD 对象，有参构造可直接指定接口、接收模式和回调函数，简化初始化流程。

示例：

```
c++
// 无参构造
ls_canfd can;

// 有参构造（使用 CAN1 接口，独立线程模式，指定回调函数）
ls_canfd can(CAN1, CANFD_MODE_THREAD, CAN_RxCallback);
```

【析构函数】

```
c++
~ls_canfd();
```

功能：销毁 LS_CANFD 对象，自动关闭 CANFD 接口、释放 socket 资源、终止接收线程（若为独立线程模式），避免资源泄漏，无需开发者手动释放资源。

说明：析构函数会自动触发资源清理，建议在程序结束时确保 LS_CANFD 对象正常析构，尤其是在多线程场景中，避免线程资源残留。

3.3 配置函数

【设置接收回调函数】

```
c++
void set_rx_callback(ls_canfd_rx_callback_t _cb);
```

参数说明：

- _cb：接收回调函数，需符合 ls_canfd_rx_callback_t 类型定义，用于处理接收到的 CANFD 数据帧。

功能：为 CANFD 接收功能设置回调函数，适用于异步信号模式和独立线程模式，当接收到数据帧时，会自动调用该回调函数处理数据。

示例：

```
c++
// 定义回调函数（处理接收到的数据帧）
void CAN_RxCallback(const ls_canfd_frame_t &frame)
{
    // 打印接收信息
    printf("接收 CANFD 数据：ID=0x%X，长度=%d，数据：",
frame.can_id, frame.len);
    for (int i = 0; i < frame.len; i++) {
        printf("%02X ", frame.data[i]);
    }
    printf("\n");
}

// 设置回调函数
can.set_rx_callback(CAN_RxCallback);
```

【设置 CANFD 波特率】

```
c++
bool set_baudrate(uint32_t arb_baud, uint32_t data_baud);
```

参数说明：

- arb_baud：仲裁段波特率（单位：bps），常用值：250000、500000（默认）、1000000
- data_baud：数据段波特率（单位：bps），常用值：1000000、2000000（默认）、4000000

功能：设置 CANFD 总线的仲裁段和数据段波特率，需在初始化前调用，若初始化后调用会返回失败。

返回值：设置成功返回 true，失败返回 false（失败原因：参数非法、已初始化、接口异常）。

示例：

```c++
// 设置仲裁段 500kbps，数据段 2Mbps（默认配置）
if (!can.set_baudrate(500000, 2000000)) {
    printf("波特率设置失败!\n");
    return -1;
}
```

【设置 CAN 帧类型】

```c++
void set_frame_type(bool is_extended);
```

参数说明：

- is_extended：true 表示使用扩展帧（CAN ID 为 29 位），false 表示使用标准帧（CAN ID 为 11 位，默认）。

功能：配置 CANFD 通信使用的帧类型，标准帧与扩展帧不可混用，需与通信对方设备保持一致，建议在初始化前设置。

示例：

```c++
// 设置使用扩展帧（29 位 CAN ID）
can.set_frame_type(true);
```

3.4 核心功能函数

【初始化 CANFD 接口】

```c++
bool init();
```

功能：初始化 CANFD 接口，完成 socket 创建、接口配置、波特率设置、接收模式初始化等操作，初始化成功后方可进行数据收发。

返回值：初始化成功返回 true，失败返回 false（失败原因：接口不存在、权限不足、波特率非法、资源分配失败）。

说明：若使用有参构造函数指定了接口、接收模式和回调函数，调用 init()即可完成全部初始化；若使用无参构造，需先调用 set_baudrate、set_rx_callback 等配置函数，再调用 init()。

示例：

```c++
// 无参构造后初始化流程
ls_canfd can;
// 配置接口、波特率、回调函数
can.set_baudrate(500000, 2000000);
can.set_rx_callback(CAN_RxCallback);
// 初始化
if (!can.init()) {
    printf("CANFD 初始化失败!\n");
    return -1;
}
printf("CANFD 初始化成功，开始通信...\n");
```

【发送 CANFD 数据帧】

```c++
bool send_frame(const ls_canfd_frame_t &frame);
```

参数说明：

- frame：CANFD 数据帧结构体，包含 CAN ID、数据长度、数据内容，数据长度需在 0~64 字节范围内。

功能：发送一帧 CANFD 数据，支持标准帧、扩展帧，数据长度可灵活设置（0~64 字节），发送成功返回 true，失败返回 false。

返回值：发送成功返回 true，失败返回 false（失败原因：未初始化、数据长度非法、接口异常、总线错误）。

示例：

```c++
// 构造 CANFD 数据帧
ls_canfd_frame_t frame;
frame.can_id = 0x123;          // CAN ID（标准帧）
frame.len = 8;                 // 数据长度 8 字节
frame.data[0] = 0x01;          // 数据内容
frame.data[1] = 0x02;
frame.data[2] = 0x03;
frame.data[3] = 0x04;
frame.data[4] = 0x05;
frame.data[5] = 0x06;
```

```
frame.data[6] = 0x07;
frame.data[7] = 0x08;

// 发送数据帧
if (can.send_frame(frame)) {
    printf("数据发送成功!\n");
} else {
    printf("数据发送失败!\n");
}
```

【接收 CANFD 数据帧（阻塞模式）】

```
c++
bool recv_frame(ls_canfd_frame_t &frame, int timeout = -1);
```

参数说明:

- frame：用于存储接收数据帧的结构体，接收成功后会填充 CAN ID、数据长度和数据内容。
- timeout：超时时间（单位：ms），默认-1 表示无限阻塞，直到接收到数据；0 表示非阻塞，立即返回；大于 0 表示等待指定时间后返回。

功能：在阻塞模式下接收 CANFD 数据帧，仅适用于 CANFD_MODE_BLOCKING 接收模式，其他模式调用会返回失败。

返回值：接收成功返回 true，失败返回 false（失败原因：未初始化、接收模式不匹配、超时、接口异常）。

示例:

```
c++
// 阻塞模式接收数据（超时时间 5000ms）
ls_canfd_frame_t recv_frame;
if (can.recv_frame(recv_frame, 5000)) {
    // 处理接收数据
    printf("阻塞模式接收: ID=0x%X, 长度=%d\n", recv_frame.can_id,
recv_frame.len);
} else {
    printf("接收超时或接收失败!\n");
}
```

【关闭 CANFD 接口】

```c++
void close_canfd();
```

功能：手动关闭 CANFD 接口，释放 socket 资源、终止接收线程（若为独立线程模式），适用于需要提前终止 CANFD 通信的场景。

说明：关闭接口后，若需重新通信，需重新调用 init()函数完成初始化。

示例：

```c++
// 关闭 CANFD 接口
can.close_canfd();
printf("CANFD 接口已关闭\n");
```

3.5 辅助函数

【获取接口状态】

```c++
bool is_init() const;
```

功能：判断 CANFD 接口是否已成功初始化，可用于避免重复初始化或未初始化就调用收发函数。

返回值：已初始化返回 true，未初始化返回 false。

【获取错误信息】

```c++
std::string get_error_msg() const;
```

功能：获取最近一次操作的错误信息，当 init()、send_frame()、recv_frame()等函数返回 false 时，可调用该函数查看具体错误原因，便于调试。

返回值：错误信息字符串，包含具体的错误描述（如“接口不存在”“波特率非法”等）。

示例：

```c++
if (!can.init()) {
    printf("初始化失败，错误原因: %s\n",
can.get_error_msg().c_str());
```

```
    return -1;
}
```

四、使用流程

使用 LS_CANFD 类实现 CANFD 通信需遵循固定流程，确保接口初始化、数据收发、资源释放的顺序正确，以下为标准使用流程及流程示意图，适配所有使用场景。

1. 创建 LS_CANFD 对象（推荐使用有参构造，直接指定接口、接收模式和回调函数）；
2. （可选）配置参数：设置波特率、帧类型、回调函数（若有参构造未指定）；
3. 调用 init()函数完成 CANFD 接口初始化，判断初始化是否成功；
4. 初始化成功后，进行数据发送或接收操作；
5. 通信结束后，可手动调用 close_canfd()关闭接口（或等待对象析构自动关闭）。

五、使用示例

以下提供 4 种常见使用场景的完整示例代码，涵盖独立线程接收、阻塞模式接收、数据发送、双接口通信，可直接修改参数后编译运行，适配龙芯 2K300/301 板卡的实际开发需求。

5.1 独立线程模式（推荐）- 收发一体

适用于多任务场景，接收线程独立运行，不影响主线程的数据发送或其他操作，搭配回调函数处理接收数据，稳定性最高。

```c++
#include "lq_canfd.hpp"
#include <stdio.h>
#include <unistd.h>

// 接收回调函数
void CAN_RxCallback(const ls_canfd_frame_t &frame)
{
    printf("[接收] ID: 0x%X, 长度: %d, 数据: ", frame.can_id,
frame.len);
    for (int i = 0; i < frame.len; i++) {
        printf("%02X ", frame.data[i]);
    }
```

```
    printf("\n");
}

int main()
{
    // 1. 创建 CANFD 对象（CAN0 接口，独立线程模式，指定回调函数）
    ls_canfd can(CAN0, CANFD_MODE_THREAD, CAN_RxCallback);

    // 2. 设置波特率（仲裁段 500kbps，数据段 2Mbps）
    if (!can.set_baudrate(500000, 2000000)) {
        printf("波特率设置失败: %s\n",
can.get_error_msg().c_str());
        return -1;
    }

    // 3. 初始化 CANFD 接口
    if (!can.init()) {
        printf("初始化失败: %s\n", can.get_error_msg().c_str());
        return -1;
    }
    printf("CANFD 初始化成功，开始收发数据...\n");

    // 4. 主线程发送数据（循环发送）
    ls_canfd_frame_t send_frame;
    send_frame.can_id = 0x123;  // 标准帧 ID
    send_frame.len = 6;         // 数据长度 6 字节
    int count = 0;

    while (1) {
        // 填充数据（递增计数）
        for (int i = 0; i < send_frame.len; i++) {
            send_frame.data[i] = (count + i) % 256;
        }

        // 发送数据
        if (can.send_frame(send_frame)) {
            printf("[发送] ID: 0x%X, 长度: %d, 数
据: %02X %02X %02X %02X %02X %02X\n",
                   send_frame.can_id, send_frame.len,
                   send_frame.data[0], send_frame.data[1],
send_frame.data[2],
                   send_frame.data[3], send_frame.data[4],
send_frame.data[5]);
```

```
        } else {
            printf("发送失败: %s\n", can.get_error_msg().c_str());
        }

        count++;
        usleep(1000000);  // 1 秒发送一次
    }

    // 5. 关闭接口（循环退出后执行，本示例未触发）
    can.close_canfd();
    return 0;
}
```

5.2 阻塞模式 - 单独接收

适用于单任务场景，主线程阻塞等待接收数据，简单直观，无需处理线程同步，适合简单的接收测试。

```
c++
#include "lq_canfd.hpp"
#include <stdio.h>

int main()
{
    // 1. 创建 CANFD 对象（CAN1 接口，阻塞模式）
    ls_canfd can(CAN1, CANFD_MODE_BLOCKING);

    // 2. 初始化接口（使用默认波特率 500kbps/2Mbps）
    if (!can.init()) {
        printf("初始化失败: %s\n", can.get_error_msg().c_str());
        return -1;
    }
    printf("CANFD 阻塞模式初始化成功，等待接收数据...\n");

    // 3. 阻塞接收数据（无限阻塞，直到接收到数据）
    ls_canfd_frame_t recv_frame;
    while (1) {
        if (can.recv_frame(recv_frame)) {
            // 处理接收数据
            printf("[阻塞接收] ID: 0x%X, 长度: %d, 数据: ",
recv_frame.can_id, recv_frame.len);
            for (int i = 0; i < recv_frame.len; i++) {
```

```
                printf("%02X ", recv_frame.data[i]);
            }
            printf("\n");
        } else {
            printf("接收失败: %s\n", can.get_error_msg().c_str());
            usleep(100000);
        }
    }

    can.close_canfd();
    return 0;
}
```

5.3 异步信号模式 - 收发分离

适用于对实时性要求不高的场景，异步信号触发接收回调，会中断主线程的 sleep，需注意线程间数据安全。

```c++
#include "lq_canfd.hpp"
#include <stdio.h>
#include <unistd.h>

// 接收回调函数
void CAN_RxCallback(const ls_canfd_frame_t &frame)
{
    printf("[异步接收] ID: 0x%X, 长度: %d, 数据: ", frame.can_id,
frame.len);
    for (int i = 0; i < frame.len; i++) {
        printf("%02X ", frame.data[i]);
    }
    printf("\n");
}

int main()
{
    // 1. 创建 CANFD 对象（CAN0 接口，异步信号模式）
    ls_canfd can(CAN0, CANFD_MODE_ASYNC);

    // 2. 设置回调函数和波特率
    can.set_rx_callback(CAN_RxCallback);
    if (!can.set_baudrate(250000, 1000000)) {
        printf("波特率设置失败: %s\n",
```

```
can.get_error_msg().c_str());
        return -1;
    }

    // 3. 初始化接口
    if (!can.init()) {
        printf("初始化失败: %s\n", can.get_error_msg().c_str());
        return -1;
    }
    printf("CANFD 异步模式初始化成功，开始工作...\n");

    // 4. 主线程执行其他任务（此处循环打印提示）
    while (1) {
        printf("主线程运行中...\n");
        sleep(1);  // 异步信号会中断该 sleep
    }

    can.close_canfd();
    return 0;
}
```

六、注意事项

1. 权限要求

- 所有 CANFD 相关操作（初始化、收发数据、调试工具）必须使用 root 权限，否则会提示权限不足，导致操作失败；
- 建议在终端中使用 sudo 命令执行程序或调试工具，避免因权限问题影响开发。

2. 接口与硬件连接

- 确保 CANFD 接口（CAN0、CAN1）与外部设备的 CAN_H、CAN_L 引脚正确连接，避免接反（接反会导致无法收发数据）；

3. 初始化与配置顺序

- 必须先设置波特率、帧类型、回调函数等参数，再调用 init()函数初始化，初始化后修改参数会返回失败；
- 无参构造创建对象后，需手动配置接口名称（通过 set_ifname 函数，未在接口说明中列出，可参考实现文件）、波特率等参数，再初始化；
- 避免重复调用 init()函数，重复初始化会导致资源冲突，需先调用 close_canfd()关闭接口，再重新初始化。

4. 接收模式选择

- 独立线程模式（CANFD_MODE_THREAD）：推荐用于多任务场景，接收线程独立运行，不影响主线程，稳定性最高；
- 阻塞模式（CANFD_MODE_BLOCKING）：适用于单任务场景，主线程阻塞等待接收，代码简单，但无法执行其他任务；
- 异步信号模式（CANFD_MODE_ASYNC）：适用于简单场景，会中断主线程sleep，不推荐用于实时性要求高的场景。

5. 数据收发规范

- 数据长度需在 0~64 字节范围内，超出范围会导致发送失败；
- 发送和接收的帧类型（标准帧/扩展帧）、波特率必须与通信对方设备保持一致，否则无法正常通信；
- 发送数据前需判断接口是否已初始化（通过 is_init()函数），避免未初始化就调用 send_frame()导致程序崩溃。

6. 线程安全

- 独立线程模式下，回调函数中处理数据时需注意线程安全，避免多线程同时操作同一资源（如全局变量）；
- 若需在回调函数中使用 printf 等打印函数，建议添加互斥锁，避免打印信息错乱。

7. 故障排查

- 当收发数据失败时，先调用 get_error_msg()函数获取错误信息，根据错误描述排查问题；
- 若无法排查，可使用调试工具（candump、cansend）测试总线连通性，确认硬件连接和接口配置是否正确；
- 若出现总线错误，可检查终端电阻是否接好、引脚是否接反、波特率是否匹配。

8. 资源释放

- 程序结束时，确保 LS_CANFD 对象正常析构，或手动调用 close_canfd()关闭接口，避免 socket 资源和线程资源泄漏；
- 多线程场景中，需确保接收线程正常终止，避免线程残留导致程序无法正常退出。

七、常见问题

Q1: 初始化失败，提示“权限不足”?

A1: 解决方案：

- CANFD 操作需要 root 权限，执行程序时添加 sudo 命令，如：sudo ./canfd_demo；
- 检查当前用户是否拥有 root 权限，若没有，切换到 root 用户（su root）后再执行程序。

Q2: 能初始化成功，但无法发送/接收数据？

A2: 检查以下几点：

- 硬件连接：确认 CAN_H、CAN_L 引脚是否接反，总线两端是否接 120Ω终端电阻；
- 参数匹配：发送端和接收端的波特率、帧类型（标准帧/扩展帧）必须完全一致；
- 总线测试：使用 candump、cansend 工具测试总线，确认总线是否正常（如 sudo ./cansend can0 123#01，同时用 sudo ./candump can0 监听）；
- 接口占用：检查是否有其他程序或工具占用了 CANFD 接口，关闭占用程序后重新测试。

Q3: 独立线程模式下，回调函数不被调用？

A3: 可能原因及解决方案：

- 未设置回调函数：确保通过有参构造或 set_rx_callback()函数设置了回调函数；
- 接收模式错误：确认创建对象时指定的接收模式为 CANFD_MODE_THREAD，而非其他模式；
- 总线无数据：使用 candump 工具确认总线上有数据发送，若没有，检查发送端是否正常；
- 线程未启动：初始化成功后，接收线程会自动启动，若未启动，检查初始化是否成功，或重启程序。

Q3: 如何切换 CANFD 接口的接收模式？

A3: 切换步骤：

1. 调用 close_canfd()函数关闭当前接口；
2. 重新创建 LS_CANFD 对象，指定新的接收模式；
3. 重新配置参数并调用 init()函数初始化，即可切换接收模式。

Q4: 多线程场景中，程序退出时出现崩溃？

A4: 原因：接收线程未正常终止，导致资源冲突；解决方案：

- 程序退出前，先调用 close_canfd()关闭接口，终止接收线程；

- 避免在主线程退出后，接收线程仍在运行，可通过全局标志位控制接收线程退出。

八、参考资源

1. CANFD 相关库文件

- 头文件：libraries/drv/inc/lq_canfd.hpp
- 实现文件：libraries/drv/src/lq_canfd.cpp

2. 示例代码

- CANFD 基础示例：example/src/06_lq_canfd_demo.cpp

3. 参考文档

- CANFD 协议参考：ISO 11898-1:2015（CANFD 协议官方标准）
- 硬件参考：龙芯 2K300/301 板卡硬件手册（含 CANFD 接口引脚定义、硬件参数等）